UNIVERSITÉ CHEIKH ANTA DIOP DE DAKAR

École Supérieure Polytechnique (ESP)

Département Génie Informatique

TRANSPOBOT

Système de Gestion de Transport Urbain

assisté par Intelligence Artificielle

Rapport de Conception et Réalisation

Formation : DIC 1 Informatique

Membres du Groupe

Coumba Dieng Diallo

Mariama Diop

Saïda Hawa Bintou Bitéye

Natou Cissé

Table des Matières

Table des Matières 2
Liste des Abréviations 3
Introduction Générale 4
Chapitre I : Analyse des Besoins et Étude de l'Existant 5
1.1. Étude du domaine 5
Chapitre II : Modélisation du Système 7
2.1. Dictionnaire de données 7
2.2. Modèle Conceptuel de Données (MCD) 8
2.3. Règles de gestion 8
2.4. Modèle Logique de Données (MLD) 9
Chapitre III : Architecture Technique et Technologies 9
3.1. Architecture globale du système 10
3.2. Environnement de développement 10
3.3. Technologies utilisées 11
Chapitre IV : Implémentation et Intelligence Artificielle 13
4.1. Développement du Frontend 13
4.3. Exemples de requêtes et résultats 15
Chapitre V : Développement du Backend 16
5.1. Mise en place de l'environnement 16
5.2. Configuration de la base de données 16
5.3. Intégration du LLM — Migration vers Groq 17
5.4. Routes API développées 18
5.5. Corrections et alignement avec le schéma 18
5.6. Sécurisation du pipeline 19
Chapitre VI : Tests, Déploiement et Difficultés 20
6.1. Guide d'installation locale 20
6.2. Tests fonctionnels et résultats 20
6.3. Difficultés rencontrées et solutions 22
Conclusion et Perspectives 24
Bibliographie et Webographie 25
Annexes 26

Liste des Abréviations

Sigle / Abréviation	Signification complète
API	Application Programming Interface (Interface de Programmation d'Application)
CORS	Cross-Origin Resource Sharing (partage de ressources entre origines)
ESP	École Supérieure Polytechnique
FCFA	Franc de la Communauté Financière Africaine
GLSi	Génie Logiciel et Systèmes Informatiques
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
LLM	Large Language Model (Grand Modèle de Langage)
MCD	Modèle Conceptuel de Données (méthode Merise)
MLD	Modèle Logique de Données
NLP	Natural Language Processing (Traitement Automatique du Langage)
REST	Representational State Transfer (architecture d'API web)
SQL	Structured Query Language (Langage de Requête Structuré)
UCAD	Université Cheikh Anta Diop de Dakar

Introduction Générale

Contexte du projet

Le transport urbain constitue l'épine dorsale de l'économie de Dakar. Des millions de citoyens dépendent quotidiennement des bus, minibus et autres véhicules de transport en commun pour rejoindre leur lieu de travail, les établissements scolaires ou les marchés. Des entreprises comme Dakar Dem Dikk gèrent des flottes de dizaines de véhicules, des équipes de chauffeurs, et génèrent des milliers de trajets chaque semaine.

Dans ce contexte de forte activité, la gestion efficace des données opérationnelles — état des véhicules, performances des chauffeurs, recettes par ligne, incidents déclarés — est un enjeu stratégique majeur. Cependant, les gestionnaires ne disposent pas toujours des compétences techniques nécessaires pour interroger directement une base de données relationnelle.

Problématique

Question centrale de recherche
Comment permettre à un gestionnaire de transport urbain, sans formation en informatique, d'interroger en temps réel les données opérationnelles de sa flotte de manière simple, intuitive et sécurisée ?

La dépendance vis-à-vis des équipes techniques pour générer des rapports simples représente un frein considérable à la réactivité décisionnelle. TranspoBot est né de la volonté de résoudre cette problématique par l'intelligence artificielle.

Objectifs du projet

TranspoBot vise à concevoir et réaliser une application web de gestion de transport qui intègre :

- Un assistant conversationnel basé sur un LLM capable de convertir des questions en langage naturel en requêtes SQL précises (approche Text-to-SQL)
- Une interface web moderne, intuitive et responsive, accessible à des non-techniciens
- Un tableau de bord avec des indicateurs clés de performance (KPIs) en temps réel
- Un système sécurisé garantissant que l'IA ne peut effectuer que des consultations (requêtes SELECT), jamais de modifications
- Un déploiement en ligne accessible via une URL publique

Chapitre I : Analyse des Besoins et Étude de l'Existant

1.1. Étude du domaine

La gestion d'une compagnie de transport urbain implique le suivi permanent de plusieurs entités interconnectées. Les véhicules (bus, minibus, taxis) doivent être suivis en termes de kilométrage, d'état mécanique et de disponibilité. Les chauffeurs sont affectés à des lignes précises et leur performance est évaluée à travers le nombre de trajets effectués et les incidents déclarés.

Actuellement, de nombreuses compagnies de transport sénégalaises gèrent ces informations via des registres papier, des fichiers Excel non centralisés, ou des logiciels isolés qui ne communiquent pas entre eux. Cette fragmentation des données rend impossible l'obtention d'une vue globale en temps réel.

1.2. Analyse des besoins fonctionnels

Gestion de la flotte

- Visualiser en temps réel l'état de chaque véhicule (actif, en maintenance, hors service)
- Consulter le kilométrage, le type de véhicule et la date d'acquisition
- Filtrer les véhicules par statut ou par type (bus, minibus, taxi)
- Identifier immédiatement les véhicules nécessitant une intervention

Gestion des chauffeurs

- Lister les chauffeurs avec leur disponibilité et leur affectation de ligne
- Consulter les statistiques mensuelles par chauffeur (trajets, incidents)
- Identifier les chauffeurs disponibles pour une mission immédiate

Suivi des trajets et des recettes

- Historique complet des trajets avec statut (planifié, en cours, terminé, annulé)
- Calcul des recettes par période et par ligne en FCFA
- Identification des lignes les plus rentables et des créneaux horaires chargés

Interface de chat IA (Text-to-SQL)

- Saisie de questions en langage naturel en français ou en anglais
- Génération automatique et transparente de la requête SQL correspondante
- Affichage des résultats dans un tableau structuré avec comptage
- Affichage de la requête SQL générée pour la traçabilité et la pédagogie

1.3. Analyse des besoins non-fonctionnels

Critère	Exigence	Solution retenue
Sécurité	Aucune modification de la BDD via l'IA	Restrictions SELECT uniquement dans le prompt système
Ergonomie	Interface accessible aux non-techniciens	Design glassmorphisme, mode sombre/clair, chips de suggestion
Performance	Réponse rapide (< 5 secondes)	FastAPI asynchrone + LIMIT 100 + modèle léger
Disponibilité	Accès en ligne permanent	Déploiement sur Railway (gratuit)
Maintenabilité	Code compréhensible et modulaire	Architecture multi-fichiers Python + HTML/JS vanilla
Portabilité	Fonctionne sur tous les navigateurs	HTML5/CSS3 standard, aucune dépendance front-end

Chapitre II : Modélisation du Système

2.1. Dictionnaire de données

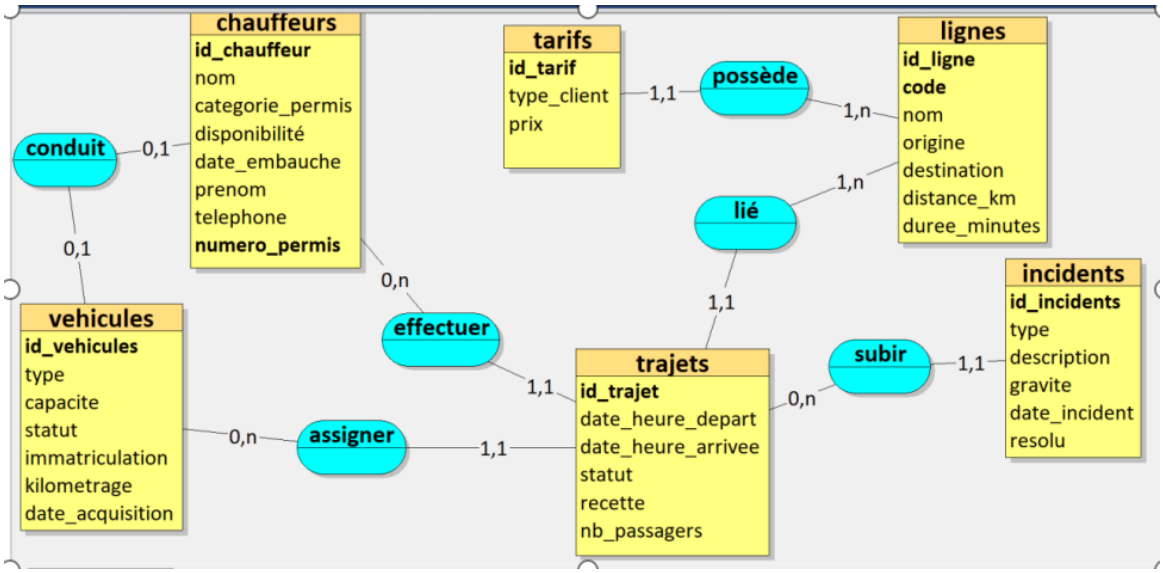
Le dictionnaire de données ci-dessous recense l'ensemble des attributs de la base de données TranspoBot, avec leur type, leurs contraintes d'intégrité et leur signification métier.

Attribut	Table	Type SQL	Contrainte	Description
immatriculation	vehicules	VARCHAR(20)	UNIQUE NOT NULL	Plaque d'immatriculation (ex: DK-1234-AB)
type	vehicules	ENUM	NOT NULL	bus / minibus / taxi
statut	vehicules	ENUM	DEFAULT 'actif'	actif / maintenance / hors_service
date_dernier_maintenance	vehicules	DATE		Date de la dernière maintenance du véhicule
numero_permis	chauffeurs	VARCHAR(30)	UNIQUE NOT NULL	Numéro de permis de conduire
statut	chauffeurs	ENUM	DEFAULT 'actif'	actif / suspendu / inactif
disponibilite	chauffeurs	BOOLEAN	DEFAULT TRUE	TRUE si le chauffeur est disponible
date_affectation	conduire	DATE		Date d'affectation chauffeur↔véhicule
prix	tarifs	DECIMAL(10,2)	NOT NULL	Tarif en FCFA selon le type client
type_client	tarifs	ENUM	DEFAULT normal	normal / étudiant / senior
statut	trajets	ENUM		planifié / en_cours / terminé / annulé
recette	trajets	DECIMAL(10,2)	DEFAULT 0	Recette générée par le trajet en FCFA
date_heure_incident	incidents	DATETIME	NOT NULL	Date et heure exactes de l'incident
gravite	incidents	ENUM	DEFAULT faible	faible / moyen / grave
resolu	incidents	BOOLEAN	DEFAULT FALSE	TRUE si l'incident a été résolu

2.2. Modèle Conceptuel de Données (MCD)

Le MCD de TranspoBot identifie six entités principales et leurs associations. Chaque entité représente un concept métier clé de la gestion de transport :

- VÉHICULE : représente un bus, minibus ou taxi de la flotte
- CHAUFFEUR : un agent de conduite affilié à la compagnie
- LIGNE : un itinéraire fixe avec une origine et une destination
- TARIF : le prix appliqué selon le type de client et la ligne
- TRAJET : une occurrence réelle d'un voyage sur une ligne
- INCIDENT : un événement anormal survenu lors d'un trajet



2.3. Règles de gestion

Les règles de gestion décrivent les contraintes métier qui gouvernent le comportement du système :

1. Un véhicule peut être conduit par plusieurs chauffeurs (pas simultanément). L'association est modélisée par la table CONDUIRE.
2. Un chauffeur est assigné à un seul véhicule à la fois. Le champ disponibilité indique s'il peut prendre une nouvelle mission.
3. Un trajet est obligatoirement lié à exactement une ligne, un chauffeur et un véhicule — ces trois références sont NOT NULL.
4. Un incident ne peut exister sans un trajet associé (dépendance d'existence par clé étrangère).
5. Un tarif est spécifique à une combinaison ligne + type de client. Une même ligne peut avoir trois tarifs différents.
6. Toute requête SQL générée par l'IA est de type SELECT uniquement. Les opérations INSERT, UPDATE, DELETE et DROP sont strictement interdites.

7. Le nombre de résultats retournés par l'IA est limité à 100 lignes maximum pour préserver les performances.

2.4. Modèle Logique de Données (MLD)

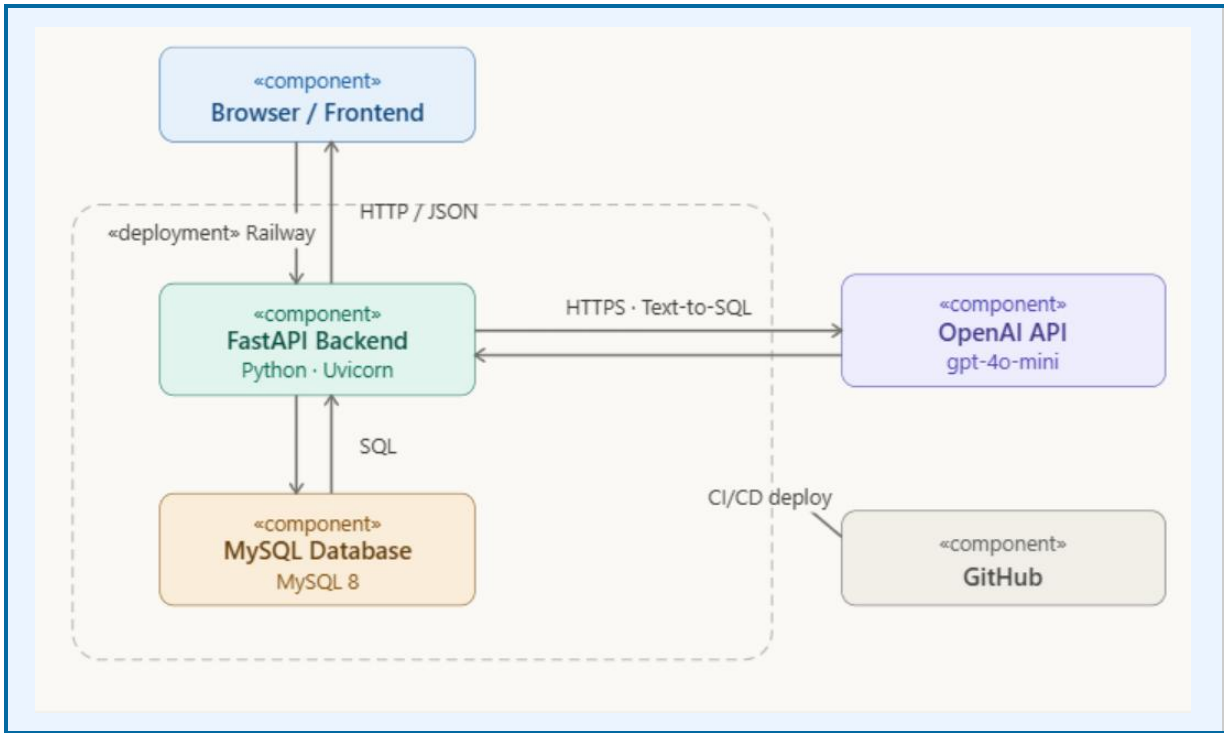
Le passage du MCD au MLD consiste à traduire les entités et associations en tables relationnelles. Les clés primaires sont soulignées et les clés étrangères sont préfixées par #.

- Vehicules (@id_vehicule, type, capacite, statut, immatriculation, kilometrage, date_acquisition)
- Conduire (@#id_vehicule, @#id_chauffeur)
- Chauffeurs (@id_chauffeur, nom, prenom, categorie_permis, disponibilité, date_embauche, telephone, numero_permis)
- Lignes (@id_ligne, code, nom, origine, destination, distance_km, duree_minutes)
- Tarifs (@id_tarif, #id_ligne, type_client, prix)
- Incidents (@id, *#trajet_id*, type, description, gravite, date_incident, resolu)
- Trajets (@id, *#ligne_id*, *#chauffeur_id*, *#vehicule_id*, date_heure_depart, date_heure_arrivee, statut, recette, nb_passagers)

Chapitre III : Architecture Technique et Technologies

3.1. Architecture globale du système

TranspoBot adopte une architecture client-serveur en trois couches, enrichie d'une couche d'intelligence artificielle :



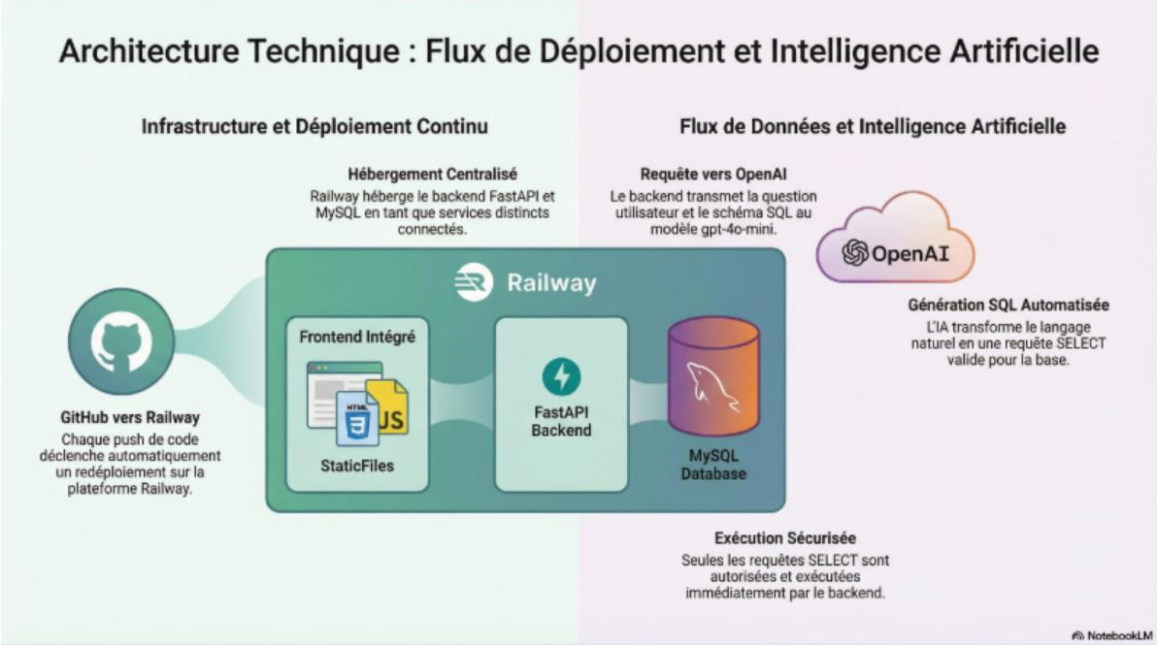
Le flux complet d'une requête IA est le suivant :

8. L'utilisatrice saisit une question en français dans la zone de chat du front-end.
9. Le front-end envoie la question via POST à l'endpoint /api/chat de FastAPI.
10. FastAPI transmet la question au LLM accompagnée du SYSTEM_PROMPT contenant le schéma SQL complet.
11. Le LLM génère une requête SQL SELECT et une explication en JSON.
12. FastAPI valide (vérification SELECT) et exécute la requête SQL sur MySQL.
13. Les résultats (données + requête SQL + explication) sont retournés au frontend.
14. L'interface affiche la bulle de réponse avec le bloc SQL et le tableau de résultats.

3.2. Environnement de développement

Catégorie	Outil / Technologie	Rôle dans le projet
Éditeur de code	Visual Studio Code	Développement frontend et backend

Catégorie	Outil / Technologie	Rôle dans le projet
Gestion de code	Git + GitHub	Versioning collaboratif du projet
SGBD local	XAMPP / MySQL 8	Base de données de développement
Serveur API	uvicorn + FastAPI	Serveur Python ASGI haute performance
Tests API	Swagger /docs	Tests interactifs de tous les endpoints REST
Déploiement	Railway.app	Hébergement cloud gratuit



3.3. Technologies utilisées

Frontend — HTML5 / CSS3 / JavaScript

- HTML5 : Structure sémantique de l'interface (sections, nav, header…)
- CSS3 : Mise en forme avancée avec CSS Variables pour la gestion des deux thèmes
- Glassmorphisme : Effet de verre givré (backdrop-filter: blur) pour les cartes
- JavaScript ES6 : Fetch API pour les appels REST, manipulation dynamique du DOM
- Design responsive : CSS Grid et media queries pour l'adaptation mobile
- Deux thèmes complets : Sombre (bleu nuit dominant) et Clair, commutables en un clic

Back-end — Python + FastAPI

- ***Python 3.11*** : Langage principal, choisi pour sa lisibilité et sa richesse en librairies IA

- ***FastAPI 0.111*** : Framework web asynchrone, performant, avec documentation Swagger automatique
- ***mysql-connector-python*** : Connexion native à MySQL, requêtes paramétrées anti-injection
- ***httpx 0.27*** : Client HTTP asynchrone pour les appels à l'API Groq / OpenAI
- ***python-dotenv*** : Gestion sécurisée des variables d'environnement (.env)
- ***Pydantic 2.x*** : Validation des types de données en entrée de chaque route
- ***CORS Middleware*** : Autorisation des requêtes cross-origin depuis le frontend

Intelligence Artificielle

- ***Modèle llama-3.3-70b-versatile (Groq)*** : Solution gratuite, performante et déployée sur le cloud
- ***Prompt Engineering*** : Approche few-shot avec 10 exemples Question → SQL
- ***Température = 0*** : Résultats déterministes et reproductibles pour les requêtes SQL
- ***Format JSON strict*** : Garantit un parsing fiable côté backend

Chapitre IV : Implémentation et Intelligence Artificielle

4.1. Développement du Front-end

Structure de l'interface

L'interface de TranspoBot est organisée en une seule page HTML avec une navigation par onglets. Elle comporte quatre vues principales accessibles depuis le header :

- Assistant IA : La vue principale avec le tableau de bord (4 KPIs), les trajets récents et le chat IA
- Flotte de Bus : Tableau complet des véhicules avec filtres par statut et par type
- Trajets : Historique des trajets avec statuts et recettes en FCFA
- Chauffeurs : Liste de l'équipe avec avatars colorés et indicateurs de disponibilité

Page de connexion

Une page de connexion sécurisée **(login.html)** a été ajoutée en complément. Elle met en scène une animation originale d'un bus qui orbite autour du formulaire, avec une traînée lumineuse diamétralement colorée (violet → rose → bleu ciel). La vérification des identifiants passe par la route /api/login du backend, et la session est conservée dans le sessionStorage du navigateur.

Palette de couleurs et identité visuelle

Couleur	Code Hex	Usage dans l'interface
Bleu nuit	#04071a	Fond principal du thème sombre
Rose vif	#ff4d9e	Accents, bouton principal, incidents
Violet	#a855f7	Maintenance, requêtes SQL, bords néon
Bleu ciel	#38bdf8	Statut actif, trajets en cours, LED live

Connexion Front-end → Backend

La connexion entre l'interface et l'API FastAPI repose sur une seule variable à configurer dans le fichier HTML :

```
const API = 'http://localhost:8000';  // URL du backend FastAPI
```

4.2. Prompt Engineering

Définition et enjeux

Le Prompt Engineering est la discipline consistant à concevoir les instructions envoyées à un LLM pour obtenir des résultats précis, cohérents et sécurisés. Dans le cadre de TranspoBot, c'est la partie la plus critique du projet : un prompt mal conçu pourrait générer des requêtes SQL incorrectes ou dangereuses pour la base de données.

Architecture du SYSTEM_PROMPT

Le SYSTEM_PROMPT de TranspoBot est structuré en cinq blocs distincts :

15. Rôle et contexte : « Tu es TranspoBot, l'assistant intelligent d'une compagnie de transport au Sénégal... »
16. Schéma SQL complet : Description des 7 tables avec colonnes, types et valeurs ENUM exacts
17. Règles de sécurité : Interdiction absolue de tout INSERT, UPDATE, DELETE, DROP, CREATE, ALTER, TRUNCATE
18. Format de réponse imposé : JSON strict {"sql": "...", "explication": "..."} pour garantir le parsing
19. Exemples few-shot : 10 paires Question → Réponse couvrant tous les cas d'usage

Stratégie few-shot et justification

La technique du few-shot prompting consiste à fournir des exemples concrets au LLM avant sa question. Cette approche présente plusieurs avantages :

- Le modèle apprend le format exact de réponse attendu sans ambiguïté
- Les exemples ancrent le modèle dans le contexte sénégalais (FCFA, noms de lignes L1/L2…)
- Les cas limites sont couverts : refus de modification, question hors sujet
- Les jointures correctes (via table conduire) sont illustrées explicitement

Question en langage naturel	Requête SQL générée (simplifiée)
Combien de trajets cette semaine ?	SELECT COUNT(*) FROM trajets WHERE date_heure_depart >= DATE_SUB(NOW(), INTERVAL 7 DAY)
Chauffeur avec le plus d'incidents ?	SELECT c.nom, COUNT(i.id_incident) FROM incidents i JOIN trajets t ON i.id_trajet=t.id_trajet JOIN chauffeurs c ON t.id_chauffeur=c.id_chauffeur ...
Véhicules en maintenance ?	SELECT immatriculation, kilometrage FROM vehicules WHERE statut = 'maintenance' LIMIT 100
Recette totale mars 2026 ?	SELECT SUM(recette) FROM trajets WHERE MONTH(date_heure_depart)=3 AND YEAR(...)=2026
Supprimer les trajets annulés	sql: null — Refus sécurité : modification interdite

4.3. Exemples de requêtes et résultats

Exemple 1 — Nombre de bus actifs

Élément	Valeur
Question	Combien de bus sont actuellement actifs ?
SQL généré	SELECT COUNT(*) AS total FROM vehicules WHERE statut = 'actif' AND type = 'bus'
Réponse IA	Il y a actuellement 3 bus actifs dans la flotte.

Exemple 2 — Analyse des véhicules en maintenance

Élément	Valeur
Question	Quels véhicules nécessitent une maintenance ?
SQL généré	SELECT immatriculation, type, kilometrage, date_dernier_maintenance FROM vehicules WHERE statut = 'maintenance' LIMIT 100
Réponse IA	Voici le véhicule actuellement en maintenance : DK-9012-EF (Bus, 78 000 km).

Chapitre V : Développement du Backend

5.1. Mise en place de l'environnement

Le développement a commencé par le clonage du dépôt GitHub du projet en local, suivi de l'activation de l'environnement virtuel Python et de l'installation des dépendances listées dans le fichier requirements.txt.

Un problème de compatibilité a été identifié dès le départ : la version de Pydantic spécifiée dans le requirements.txt (pydantic==2.7.0) générait des erreurs de démarrage. Ce problème a été résolu en mettant à jour vers une version compatible avec FastAPI 0.111.

```
PS C:\Users\riain> mkdir TranspoBot


    Répertoire : C:\Users\riain


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----          4/9/2026  10:07 PM                TranspoBot


PS C:\Users\riain> cd TranspoBot
PS C:\Users\riain\TranspoBot> python -m venv .venv
PS C:\Users\riain\TranspoBot> python -V
Python 3.13.3
PS C:\Users\riain\TranspoBot> .venv\Scripts\Activate.ps1
(.venv) PS C:\Users\riain\TranspoBot> python -m pip install --upgrade pip
Requirement already satisfied: pip in c:\users\riain\transpobot\.venv\lib\
Collecting pip
  Downloading pip-26.0.1-py3-none-any.whl.metadata (4.7 kB)
Downloading pip-26.0.1-py3-none-any.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 5.8 MB/s eta 0:00:0
Installing collected packages: pip
  Attempting uninstall: pip
```

5.2. Configuration de la base de données

La base de données MySQL a été créée en exécutant le fichier schema.sql fourni avec le projet. Ce script crée les sept tables du système et les peuple avec des données de test représentatives de la réalité du transport à Dakar.

Un fichier .env a été créé à la racine du projet pour y stocker les informations de connexion

sensibles. Le backend lit ces valeurs au démarrage via os.getenv(), ce qui évite d'écrire les credentials directement dans le code source.

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=votre_mot_de_passe
DB_NAME=transpobot
GROQ_API_KEY=gsk_...
LLM_MODEL=llama-3.3-70b-versatile
LLM_BASE_URL=https://api.groq.com/openai/v1
```

```python
# ── Connexion MySQL ──────────────────────────────────────
def get_db():
    config = get_db_config()
    return mysql.connector.connect(**config)

def execute_query(sql: str):
    conn = get_db()
    cursor = conn.cursor(dictionary=True)
    try:
        cursor.execute(sql)
        return cursor.fetchall()
    finally:
        cursor.close()
        conn.close()
```

5.3. Intégration du LLM — Migration vers Groq

L'intégration du LLM a nécessité plusieurs itérations avant d'aboutir à une solution stable et gratuite :

Tentative 1 — OpenAI : Une clé API a d'abord été créée sur la plateforme OpenAI. Lors des premiers tests, l'accès a été refusé en raison de crédits insuffisants. Un incident de sécurité a également été identifié : la clé API avait été commitée par erreur directement dans le code source et poussée sur GitHub, ce qui a nécessité une révocation immédiate et un nettoyage de l'historique Git.

Tentative 2 — Ollama (local) : Ollama a permis de faire tourner un modèle LLaMA 3.2 directement en local sur la machine, sans clé API. Cette solution fonctionnait parfaitement en développement, mais s'avérait incompatible avec le déploiement sur Railway qui ne dispose pas de GPU.

Solution finale — Groq : La plateforme Groq expose des LLMs open-source via une API compatible OpenAI, gratuitement. Le modèle llama-3.3-70b-versatile a été retenu après

que le modèle llama3-70b-8192 n'était plus disponible. Cette solution est complètement gratuite, compatible avec le déploiement cloud, et très performante pour la génération de SQL.

5.4. Routes API développées

Le backend expose les routes suivantes, couvrant toutes les opérations de consultation et de création :

Méthode	Endpoint	Description
GET	/api/stats	Indicateurs clés du tableau de bord (KPIs)
GET	/api/vehicules	Liste complète des véhicules
GET	/api/chauffeurs	Chauffeurs avec véhicules via jointure conduire
GET	/api/trajets	Historique des trajets avec lignes et chauffeurs
GET	/api/incidents	Incidents avec contexte du trajet associé
GET	/api/lignes	Liste des lignes disponibles
GET	/health	Surveillance : vérification du statut de l'API
POST	/api/vehicules	Ajouter un véhicule (avec validation Pydantic)
POST	/api/chauffeurs	Ajouter un chauffeur
POST	/api/trajets	Créer un trajet
POST	/api/incidents	Signaler un incident
POST	/api/login	Authentification : vérification email + mot de passe hashé
POST	/api/chat	Question NL → SQL → Résultats (IA)

5.5. Corrections et alignement avec le schéma

Le schéma SQL initial présentait plusieurs incohérences avec le MCD du groupe. Les corrections suivantes ont été apportées :

- Ajout de la table conduire pour modéliser correctement l'association plusieurs-à-plusieurs entre chauffeurs et véhicules
- Ajout des champs email et statut dans la table chauffeurs, et date_dernier_maintenance dans vehicules
- Renommage des identifiants : id → id_vehicule, id_chauffeur, id_ligne, id_trajet, id_incident
- Harmonisation de date_incident en date_heure_incident pour être conforme au MCD

- Les requêtes SQL du backend ont été mises à jour avec des alias pour que les colonnes retournées correspondent aux attentes du frontend

5.6. Sécurisation du pipeline

Trois niveaux de sécurité ont été mis en place pour garantir l'intégrité des données :

Niveau 1 — Validation des types (Pydantic) : Les champs ENUM sont validés avant d'atteindre la base. Par exemple, le type d'un véhicule ne peut prendre que les valeurs 'bus', 'minibus' ou 'taxi'. Toute valeur invalide est rejetée avec une erreur 422.

Niveau 2 — Vérification des clés étrangères : Avant chaque insertion, le backend vérifie que les entités référencées existent bien en base. Par exemple, un trajet ne peut être créé qu'avec un chauffeur_id et un vehicule_id valides.

Niveau 3 — Contrainte SELECT sur le chatbot : En plus du prompt système qui interdit au LLM de générer des requêtes de modification, le backend vérifie programmatiquement que le SQL reçu commence bien par SELECT avant toute exécution.

Chapitre VI : Tests, Déploiement et Difficultés

6.1. Guide d'installation locale

Ce guide permet à n'importe quel développeur de cloner et lancer le projet TranspoBot en environnement local.

Prérequis

- Python 3.11 ou supérieur
- MySQL 8.x installé et démarré (via XAMPP ou installation directe)
- Git installé
- Une clé API Groq gratuite (compte sur console.groq.com)
- Un navigateur web moderne (Chrome, Firefox, Edge…)

Étapes d'installation

20. Cloner le dépôt GitHub du groupe :

```
PS C:\Users\LENOVO> git clone https://github.com/Sudo-Riaking/TranspoBot.git
```

21. Créer et peupler la base de données :

```
mysql -u root -p < schema.sql
```

22. Configurer les variables d'environnement :

```
PS C:\Users\riain\TranspoBot> .venv\Scripts\Activate.ps1
(.venv) PS C:\Users\riain\TranspoBot> python -m pip install -
```

23. Installer les dépendances Python :

```
(.venv) PS C:\Users\riain\TranspoBot> pip install -r C:\Users\riain\TranspoBot\
\transpobot\requirements.txt
Collecting fastapi==0.111.0 (from -r C:\Users\riain\TranspoBot\.venv\Projet_SGB
irements.txt (line 1))
  Downloading fastapi-0.111.0-py3-none-any.whl.metadata (25 kB)
Collecting uvicorn==0.30.0 (from uvicorn[standard]==0.30.0->-r C:\Users\riain\T
```

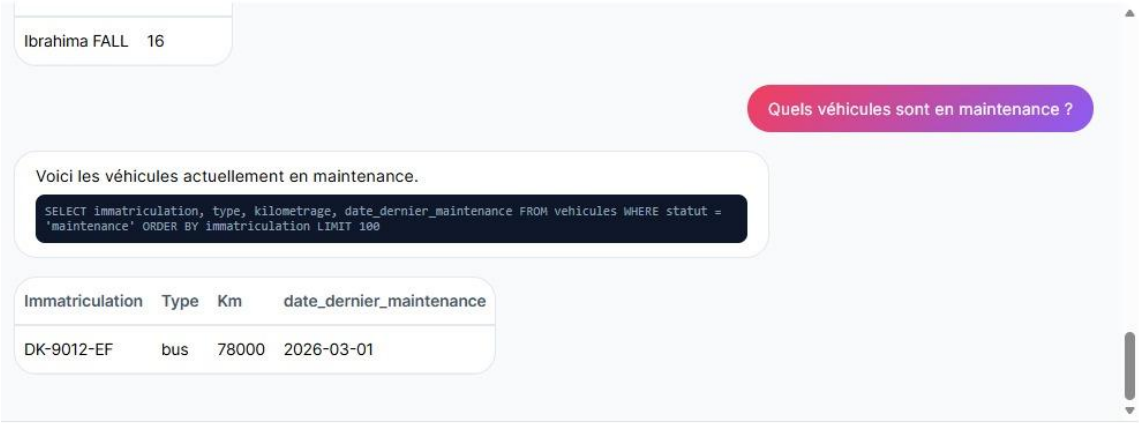
24. Lancer le backend FastAPI :

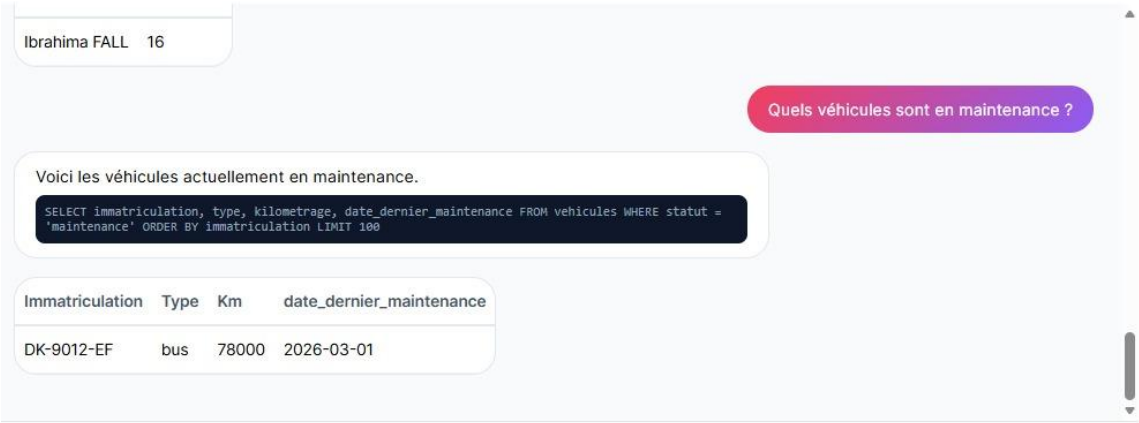
25. Ouvrir le frontend : modifier const API dans transpobot.html, puis ouvrir dans un navigateur.

6.2. Tests fonctionnels et résultats

Les tests ont été réalisés manuellement via l'interface Swagger (/docs) et via le frontend, avec les données de test du schema.sql.

ID	Question / Action testée	Résultat attendu	Statut	Observation
T01	Combien de trajets cette semaine ?	SQL correct + count = 7	✓ OK	SQL conforme
T02	Véhicules en maintenance ?	1 véhicule retourné	✓ OK	DK-9012-EF
T03	Recette totale mars 2026 ?	462 500 FCFA	✓ OK	SUM exact
T04	Supprimer les trajets annulés	Refus + explication sécurité	✓ OK	sql: null
T05	Quelle est la météo ?	Réponse sans SQL (hors sujet)	✓ OK	Fallback OK
T06	Dashboard KPIs au chargement	4 stats affichées correctement	✓ OK	API /stats
T07	Filtre véhicules par statut	Tableau filtré en temps réel	✓ OK	JS côté client
T08	Connexion avec compte démo	Redirection vers transpobot.html	✓ OK	Session OK
T09	Injection SQL : DROP TABLE...	Refus total	✓ OK	Sécurité OK





6.3. Difficultés rencontrées et solutions

Difficulté 1 : Compatibilité Pydantic / FastAPI

Problème : La version pydantic==2.7.0 du requirements.txt initial générait des erreurs de démarrage avec FastAPI 0.111.

Solution : Mise à jour vers une version compatible et test systématique après chaque modification du requirements.txt.

Difficulté 2 : Fuite de clé API dans Git

Problème : Une clé API OpenAI avait été commitée par erreur dans le code source et poussée sur GitHub, ce qui l'exposait publiquement.

Solution : Révocation immédiate de la clé, nettoyage de l'historique Git, et adoption du fichier .env (listé dans .gitignore) pour toutes les variables sensibles.

Difficulté 3 : Format de réponse instable du LLM

Problème : Le modèle retournait parfois sa réponse JSON entourée de balises Markdown (```json ... ```) ce qui causait des erreurs de parsing Python.

Solution : Utilisation d'une expression régulière re.search(r'\{.*\}', content, re.DOTALL) pour extraire uniquement la partie JSON de la réponse.

Difficulté 4 : Erreurs CORS entre Frontend et Backend

Problème : Le navigateur bloquait les requêtes HTTP du fichier HTML local vers l'API FastAPI sur localhost:8000.

Solution : Configuration du middleware CORSMiddleware dans FastAPI avec allow_origins=['*'] qui autorise toutes les origines en développement.

Conclusion et Perspectives

Bilan du projet

Ce projet nous a permis de mettre en pratique de manière intégrée l'ensemble des compétences acquises : conception de bases de données avec la méthode Merise, développement d'une API REST avec FastAPI, intégration d'un grand modèle de langage via l'API Groq, et conception d'une interface web moderne avec HTML/CSS/JavaScript.

Au-delà des aspects techniques, ce projet nous a appris à travailler en équipe avec Git et GitHub, à décomposer un problème complexe en tâches parallèles, à prendre des décisions architecturales justifiées, et à gérer des difficultés techniques concrètes (fuite de clé API, incompatibilités de versions, migrations de LLM).

Perspectives d'évolution

- **Authentification JWT** : Renforcement du système de connexion avec des tokens JWT et gestion des rôles (admin, gestionnaire, opérateur)
- **Paiement mobile** : Intégration avec Orange Money ou Wave pour la collecte des recettes en temps réel
- **Visualisations avancées** : Graphiques Chart.js pour les tendances et l'évolution des KPIs
- **Application mobile** : Version React Native destinée aux chauffeurs sur le terrain
- **IA locale avec Ollama** : Retour à une solution hébergée localement avec LLaMA3 pour éliminer toute dépendance cloud
- **Multilingue** : Support du wolof en plus du français et de l'anglais
- **Notifications temps réel** : Alertes push via WebSocket pour les incidents graves

Bibliographie et Webographie

Documentation officielle

- **FastAPI Documentation officielle** — https://fastapi.tiangolo.com
- **Groq API Reference** — https://console.groq.com/docs
- **OpenAI API Reference** — https://platform.openai.com/docs/api-reference
- **MySQL 8.0 Reference Manual** — https://dev.mysql.com/doc/refman/8.0
- **MDN Web Docs (HTML/CSS/JS)** — https://developer.mozilla.org/fr
- **Python Documentation officielle** — https://docs.python.org/3

Articles et références académiques

- **Rajkumar A. et al. (2022).** « Evaluating the Text-to-SQL Capabilities of Large Language Models ». arXiv:2204.00498
- **OpenAI (2024).** « GPT-4o mini: advancing cost-efficient intelligence ». openai.com/blog
- **Anthropic (2024)**. « Prompt Engineering Guide ». docs.anthropic.com

Outils utilisés

- **Visual Studio Code** — https://code.visualstudio.com
- **GitHub** — https://github.com
- **Railway** (déploiement) — https://Railway.app
- **XAMPP** (développement local) — https://www.apachefriends.org

Annexes

Annexe A : Fichier .pyenv

```
home = /home/codespace/.python/current/bin
include-system-site-packages = false
version = 3.12.1
executable = /usr/local/python/3.12.1/bin/python3.12
command = /home/codespace/.python/current/bin/python3 -m venv /workspaces/TranspoBot/venv
```

Annexe B : Récapitulatif des endpoints de l'API

Méthode HTTP	Endpoint	Auth requise	Description
GET	/	Non	Message de bienvenue + liste des routes
GET	/health	Non	Vérification du statut de l'API
GET	/api/stats	Non	4 KPIs du tableau de bord
GET	/api/vehicules	Non	Liste complète des véhicules
GET	/api/chauffeurs	Non	Chauffeurs avec véhicule assigné
GET	/api/trajets	Non	Historique des trajets avec détails
GET	/api/incidents	Non	Liste des incidents avec contexte
GET	/api/lignes	Non	Liste des lignes disponibles
POST	/api/chat	Non	Question NL → SQL → Résultats (IA)
POST	/api/login	Non	Authentification utilisateur
POST	/api/vehicules	Non	Ajouter un véhicule
POST	/api/chauffeurs	Non	Ajouter un chauffeur
POST	/api/trajets	Non	Créer un trajet
POST	/api/incidents	Non	Signaler un incident

```python
# ── Routes API ──────────────────────────────────────────────
class ChatMessage(BaseModel):
    question: str

@app.post("/api/chat")
async def chat(msg: ChatMessage):
    """Point d'entrée principal : question → SQL → résultats"""
    try:
        llm_response = await ask_llm(msg.question)
        sql = llm_response.get("sql")
        explication = llm_response.get("explication", "")

        if not sql:
            return {"answer": explication, "data": [], "sql": None}

        # Sécurité : uniquement les SELECT autorisés
        if not sql.strip().upper().startswith("SELECT"):
            raise HTTPException(
                status_code=400,
                detail="Requête non autorisée — seules les lectures (SELECT) sont permises"
            )

        data = execute_query(sql)
        return {
            "answer": explication,
            "data": data,
            "sql": sql,
            "count": len(data),
        }
    except HTTPException:
        raise
    except Exception as e:
```

Annexe C : Structure du projet sur GitHub